Semáforo de reseñas

Clasificación de sentimiento en español con modelos de lenguaje

Estudiante: Pedro Bueno Pereyra **Curso:** IFTS N.º 18 **Fecha:** 16/10/2026

A. El problema

1. ¿Qué problema querés resolver y para quién? ¿Por qué un modelo de IA y no unas reglas escritas a mano (un if con palabras clave)?

El problema es revisar muchas opiniones de clientes para detectar rápidamente si el tono de una reseña parece positivo, neutral o negativo. La herramienta está pensada como apoyo exploratorio para quien gestiona un comercio; no toma decisiones sobre clientes. Un conjunto pequeño de reglas por palabras clave podría servir para vocabulario fijo, pero falla con negaciones, ironía, contexto y expresiones como “Genial, otra vez me cancelaron...”. Un modelo preentrenado aporta patrones lingüísticos aprendidos, aunque también puede equivocarse; por eso su salida se presenta como orientación y no como verdad.

B. El modelo

2. ¿Qué modelos consideraste y por qué te quedaste con uno? Citá lo que leíste en su model card: tarea, etiquetas, idiomas, datos de entrenamiento, tamaño de los pesos.

Modelo	Model card y tamaño informado
cardiffnlp/twitter-xlm-roberta-base-sentiment	Clasificación de sentimiento; POSITIVE, NEUTRAL, NEGATIVE; ocho idiomas, incluido español. La tarjeta describe preentrenamiento con unos 198 millones de tweets y ajuste multilingüe de sentimiento. Pesos: 1.112.271.561 bytes (≈1,04 GiB).
pysentimiento/robertuito-sentiment-analysis	Clasificación de sentimiento en español; POS, NEU, NEG. La tarjeta describe preentrenamiento de RoBERTuito con más de 500 millones de tweets en español y ajuste con TASS 2020 (aprox. 5.000 tweets). Pesos: 435.189.428 bytes (≈415 MiB); uso no comercial indicado.

Se eligió Cardiff para la app porque admite español y se integró con la Space ZeroGPU disponible. La prueba didáctica dio 8/10 aciertos para cada modelo; esa muestra pequeña no demuestra que Cardiff sea más exacto. Se priorizaron compatibilidad de despliegue y tamaño frente a afirmar una superioridad medida.

Fuentes: model cards enlazadas al final. Las tarjetas no describen con el mismo detalle todos los corpus de ajuste; no se atribuyen datos no publicados por sus autores.

B. El modelo

3. ¿Dónde se equivoca? Mostrá ejemplos reales de tu notebook y explicá por qué creés que se equivoca, relacionándolo con los datos con los que fue entrenado.

Se probaron diez frases con etiquetas manuales: ejemplos claros, neutralidad, negación, ironía, una reseña en inglés, texto muy corto y jerga rioplatense. La tabla del notebook muestra frase, etiqueta esperada y predicha, puntaje y acierto. Es una muestra pequeña, didáctica y no generalizable.

Resultados medidos en diez frases:

Método/modelo	Aciertos / 10	Exactitud	Tiempo
Cardiff XLM-R	8 / 10	80 %	0,569 s / 10
RoBERTuito	8 / 10	80 %	0,529 s / 10

La carga/descarga medida fue 110,840 s para Cardiff y 92,619 s para RoBERTuito; depende de equipo, conexión y caché.

En una prueba separada con la frase “No me respondieron el reclamo y nadie se hizo cargo”, pipeline clasificó como negative (puntaje 0,7448) en 0,0593 s; la Space devolvió la misma etiqueta (0,7448) en 2,2485 s. El tiempo remoto incluye la llamada por red y no es directamente comparable con la inferencia local ya cargada. La Space usa Gradio nativo porque ZeroGPU requiere una función Gradio marcada con @spaces.GPU; por eso la llamada remota se hace con gradio_client y no con InferenceClient. La salida de pipeline y la respuesta de gradio_client pueden tener tipos diferentes; el notebook los inspecciona y normaliza para compararlos.

Frase / modelo	Referencia	Predicción	Interpretación
Cardiff: “Esperé una hora y me entregaron el pedido frío.”	Negativo	Neutro	No detectó el sentimiento negativo.
RoBERTuito: “La comida estaba bien, sin nada especial.”	Neutro	Positivo	Interpretó una valoración tenue como positiva.

Cardiff puede confundir una reseña de comercio con patrones de tweets multilingües: la queja “pedido frío” expresa el problema por su consecuencia sin una palabra negativa directa. RoBERTuito fue preentrenado con tweets en español y ajustado con TASS, pero “bien, sin nada especial” es una valoración tenue que puede quedar fuera de sus patrones aprendidos. En ambos casos, el puntaje expresa confianza del modelo, no garantía de acierto.

4. Tu app, ¿está entrenando o infiriendo? Explicá la diferencia con tu propio caso.

La app infiere: aplica los pesos ya entrenados de Cardiff a una reseña nueva y devuelve una etiqueta y un puntaje. No ajusta sus parámetros con cada texto. Entrenar implicaría actualizar esos parámetros mediante datos y optimización, algo que no sucede al presionar “Analizar reseña”.

C. Dónde corre cada cosa

5. ¿Dónde vive el código y dónde vive el modelo? Hacé un esquema simple (cajas y flechas, a mano o en cualquier herramienta) con los lugares que intervienen en tu app: tu compu, GitHub, la plataforma de deploy y Hugging Face. En cada caja anotá qué hay ahí, y marcá con flechas qué viaja de un lugar a otro cuando alguien usa la app. Tiene que quedar claro: ¿dónde está guardado tu código? ¿Dónde se ejecuta? ¿Dónde están los pesos del modelo y dónde se hace la inferencia?

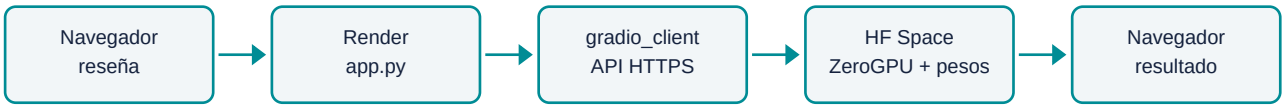
Etapa	Servicio / responsabilidad
Código fuente	GitHub conserva app.py y dependencias; el deploy toma el código del repositorio.
Interfaz	Render ejecuta app.py y sirve Gradio desde un proceso web.
Pesos e inferencia	La Space de Hugging Face aloja Cardiff; ZeroGPU asigna la GPU durante la función de clasificación.
Flujo	Navegador → Render → API Gradio/gradio_client → Space ZeroGPU → etiqueta y puntaje → Render → navegador.

El navegador envía el texto; los pesos no se trasladan: la Space procesa la solicitud y devuelve solo la predicción.

Publicación del código



Uso de la app: viaja la reseña y vuelve la predicción



6. ¿Usaste pipeline o InferenceClient en la app publicada? Justificalo con números (memoria del plan gratuito, peso del modelo, tiempo de respuesta). ¿Qué pasaría si hubieras elegido la otra?

La app publicada no usa pipeline ni InferenceClient: Render llama con gradio_client a la API Gradio de la Space. Se eligió porque ZeroGPU requiere un handler Gradio decorado con @spaces.GPU; InferenceClient no invoca directamente ese endpoint Gradio. No se presentan ambos clientes como equivalentes.

Cardiff pesa 1.112.271.561 bytes (≈1,04 GiB); la instancia Free de Render dispone de 512 MB RAM según la configuración consultada, menos que los pesos sin contar Python y Gradio. La carga local de Cardiff tomó 110,840 s; luego una inferencia ya cargada tardó 0,0593 s. La llamada a la Space tardó 2,2485 s incluyendo red/cola. Estos tiempos no comparan hardware equivalente. Si se usara pipeline en Render, el modelo no cabría en esa memoria. InferenceClient requeriría publicar en un endpoint/proveedor compatible; no llamaría a esta Space Gradio y sus créditos/costos dependerían de ese servicio.

La app valida HTTPS y el dominio .hf.space; el token de lectura se configura como secreto en Render, nunca en el repositorio.

D. El deploy

7. ¿Dónde deployaste y por qué ahí? Compará tu elección con al menos otras dos opciones vistas en clase (hosting compartido, VPS, PaaS, serverless, nubes elásticas) y explicá por qué las descartaste. ¿Por qué tu app necesita un servidor que quede encendido?

Render es un PaaS: construye desde GitHub y ejecuta el servidor web sin que tenga que administrar un sistema operativo. Una VPS se descartó porque requiere mantener sistema, actualizaciones, seguridad y escalado.

Ejecutarla solo en la computadora local no ofrece un enlace público persistente y deja de responder cuando se apaga. Serverless podría atender funciones puntuales, pero sumaría límites de ejecución y otra integración para una interfaz web que llama a un modelo remoto.

El navegador necesita que un proceso web reciba HTTP en la URL pública; ese proceso, a su vez, llama a la Space para inferir. El servicio Free de Render se suspende tras 15 minutos sin tráfico y puede tardar alrededor de un minuto en despertar, por lo que no queda siempre encendido. Para disponibilidad garantizada habría que pagar un servicio siempre activo y dimensionarlo.

E. Lo que se rompió

8. Documentá al menos dos errores reales que te aparecieron durante el trabajo. Para cada uno: el mensaje de error completo; dónde apareció; qué lo causaba; y cómo lo resolviste, paso a paso.

Error 1 — Hugging Face Space, al iniciar:
`No @spaces.GPU function detected during startup`.
Causa: ZeroGPU no reconocía la función GPU cuando Gradio estaba montado en una app FastAPI/Uvicorn personalizada.
Resolución: 1) pasar la Space a Gradio nativo; 2) conectar directamente la función decorada con `@spaces.GPU`; 3) publicar los archivos y revisar los logs de arranque.

Error 2 — Render, durante el inicio:
`ValueError: When localhost is not accessible, a shareable link must be created. Please set share=True or check your proxy settings to allow access to localhost.`
Causa: `demo.launch()` intentaba servir Gradio directamente detrás del proxy.
Resolución: 1) no activar `share=True`; 2) montar Gradio en FastAPI; 3) iniciar Uvicorn en `0.0.0.0` y en el puerto de Render `PORT`; 4) volver a desplegar y probar la URL.

Error 3 — Render, al servir la página:
`TypeError: cannot use 'tuple' as a dict key (unhashable type: 'dict')`.
Causa: incompatibilidad de versiones web al ejecutar con Python 3.14.
Resolución: fijar Python 3.13.4 y versiones compatibles de FastAPI/Starlette/Uvicorn; reconstruir y verificar que la página responda.

9. Límites de tu solución: ¿qué pasa si la usan 1000 personas a la vez? ¿Si Hugging Face cambia las condiciones del plan gratuito (como ya pasó con los Spaces)? ¿Qué cambiarías para que fuera un producto real?

Con 1.000 solicitudes simultáneas no se puede garantizar respuesta: Render puede saturarse y ZeroGPU es compartido, con cuota y cola; se pueden acumular esperas y vencer los timeouts. Hoy no hay pruebas de carga, cola propia, escalado ni SLA. Si Hugging Face cambia o retira el plan gratuito, la Space podría suspenderse o requerir pago; habría que migrar a un endpoint o servidor con presupuesto y repetir las pruebas. Para un producto real agregaría pruebas de carga, rate limits, métricas/alertas, colas, capacidad escalable, política de privacidad y presupuesto. El puntaje no está calibrado y no debe usarse para decisiones automáticas sobre personas.

F. Vocabulario

10. Tomá 5 líneas de tu app.py y señalá en ellas: una clase, una instancia, un método, un atributo, un parámetro y su argumento, y un valor de retorno.

```
L23 def clasificar_resena(texto: str) -> tuple[str, str]:
L51 cliente = Client(
L52 src=endpoint_url,
L57 predicciones = cliente.predict(texto, api_name="/clasificar")
L75 return etiqueta, f"{mejor['score']:.1%}"
```

Clase: Client. **Instancia:** cliente. **Método:** predict. **Atributo/clave:** mejor["score"]. **Parámetro:** src; **argumento:** endpoint_url. **Parámetro de función:** texto; **argumento en la llamada:** texto. **Retorno:** la tupla etiqueta y puntaje formateado.

Enlaces y declaración de uso de IA

Elemento	URL / evidencia
Aplicación pública de Render	Abrir app de Render
Space ZeroGPU de Hugging Face	Abrir Space en Hugging Face
Repositorio con código y notebook	Abrir carpeta TP_Integrador en GitHub
Capturas de clasificación positiva y negativa	Incluidas en la sección de evidencias.

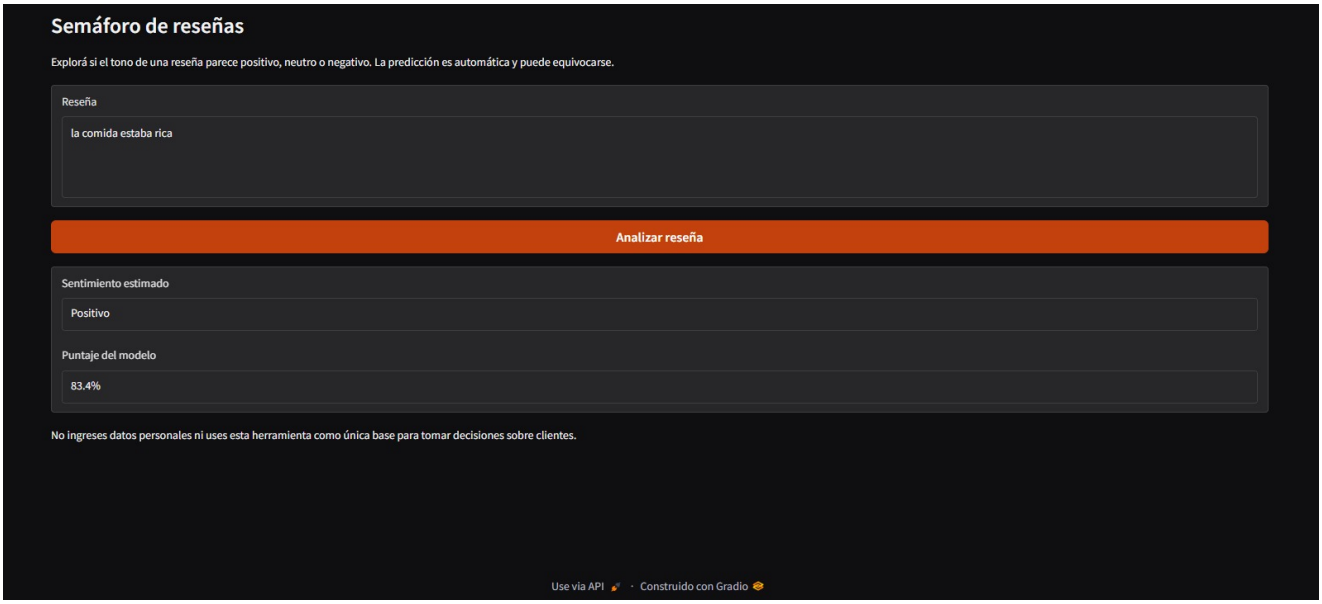
Fuentes consultadas

• Modelo Cardiff: https://huggingface.co/cardiffnlp/twitter-xlm-roberta-base-sentiment
• Modelo RoBERTuito: https://huggingface.co/pysentimiento/robertuito-sentiment-analysis
• Hugging Face Spaces: https://huggingface.co/docs/hub/spaces-overview
• Gradio Python Client: https://www.gradio.app/docs/python-client/introduction
• Render Free: https://render.com/docs/free
• Render compute plans: https://render.com/docs/compute-plans
Revisar licencias y condiciones vigentes antes de entregar.

G. Evidencia de la aplicación publicada

Las siguientes capturas muestran dos pruebas reales realizadas en la app publicada: una reseña positiva y otra negativa. También se probó el envío vacío; la interfaz respondió con un aviso para escribir una reseña antes de analizarla.

Prueba positiva: “la comida estaba rica” → Positivo (83,4 %)



Prueba negativa: “se tardaron en atenderme” → Negativo (83,5 %)

